

# Outros Tipos de Dados

## Tuplas, Sets e Dicionários

Ana Júlia Lima



Desenvolvido para  
Young 1

Ctrl Play Santa Mônica  
Vila Velha, Brasil  
Fevereiro 2026

# Sumário

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Tuplas</b>                                           | <b>2</b> |
| 1.1      | Para que servem? Quando usar?                           | 2        |
| 1.2      | Tuplas são mutáveis ou imutáveis?                       | 2        |
| 1.3      | Construindo tuplas                                      | 2        |
| 1.4      | Acessando valores                                       | 2        |
| 1.5      | Funções e métodos                                       | 2        |
| 1.5.1    | len()                                                   | 2        |
| 1.5.2    | index()                                                 | 2        |
| 1.5.3    | count()                                                 | 3        |
| 1.6      | Tentando modificar uma tupla                            | 3        |
| <b>2</b> | <b>Conjuntos (Sets)</b>                                 | <b>3</b> |
| 2.1      | Para que servem?                                        | 3        |
| 2.2      | Sets são mutáveis ou imutáveis?                         | 3        |
| 2.3      | Criando sets                                            | 3        |
| 2.4      | Adicionando elementos                                   | 3        |
| <b>3</b> | <b>Boolean e None</b>                                   | <b>4</b> |
| 3.1      | Boolean                                                 | 4        |
| 3.2      | None                                                    | 4        |
| <b>4</b> | <b>Dicionários</b>                                      | <b>4</b> |
| 4.1      | Para que servem?                                        | 4        |
| 4.2      | Dicionários são mutáveis ou imutáveis?                  | 4        |
| 4.3      | Criando dicionários                                     | 4        |
| 4.4      | Acessando itens                                         | 4        |
| 4.5      | Flexibilidade dos dicionários                           | 5        |
| 4.6      | Alterando valores                                       | 5        |
| 4.7      | Adicionando novos itens                                 | 5        |
| 4.8      | Métodos importantes                                     | 5        |
| 4.8.1    | keys()                                                  | 5        |
| 4.8.2    | values()                                                | 5        |
| 4.8.3    | items()                                                 | 5        |
| <b>5</b> | <b>Erros Comuns com Tuplas, Sets e Dicionários</b>      | <b>6</b> |
| 5.1      | 1. Tentar modificar uma tupla                           | 6        |
| 5.2      | 2. Usar métodos de lista em tuplas                      | 6        |
| 5.3      | 3. Esperar que sets mantenham ordem                     | 6        |
| 5.4      | 4. Colocar elementos repetidos em sets                  | 6        |
| 5.5      | 5. Acessar dicionário como lista                        | 6        |
| 5.6      | 6. Esquecer que chaves precisam existir                 | 7        |
| 5.7      | Resumo                                                  | 7        |
| <b>6</b> | <b>Comparação entre Estruturas de Dados</b>             | <b>7</b> |
| <b>7</b> | <b>Lista de Exercícios — Tuplas, Sets e Dicionários</b> | <b>8</b> |
| <b>8</b> | <b>Gabarito — Tuplas, Sets e Dicionários</b>            | <b>9</b> |

# 1 Tuplas

## 1.1 Para que servem? Quando usar?

Uma tupla é uma estrutura de dados muito parecida com listas.

A principal diferença é que as tuplas são **imutáveis**.

Isso significa que depois de criadas, seus valores **não podem ser alterados**.

Tuplas são usadas quando:

- queremos garantir que os dados não mudem
- representar coordenadas
- armazenar dados fixos

## 1.2 Tuplas são mutáveis ou imutáveis?

Tuplas são **imutáveis**.

Depois de criadas, não é possível adicionar, remover ou alterar elementos.

## 1.3 Construindo tuplas

Tuplas usam parênteses.

```
numeros = (10, 20, 30)
```

```
print(numeros)
# Saída: (10, 20, 30)
```

## 1.4 Acessando valores

O acesso funciona da mesma forma que listas.

```
numeros = (10, 20, 30)
```

```
print(numeros[0]) # 10
print(numeros[2]) # 30
```

## 1.5 Funções e métodos

### 1.5.1 len()

```
tupla = (1, 2, 3, 4)
```

```
print(len(tupla))
# Saída: 4
```

### 1.5.2 index()

Mostra o índice de um elemento.

```
tupla = (10, 20, 30)
```

```
print(tupla.index(20))
# Saída: 1
```

### 1.5.3 count()

Conta quantas vezes um elemento aparece.

```
tupla = (1, 2, 2, 3)
```

```
print(tupla.count(2))
```

```
# Saída: 2
```

## 1.6 Tentando modificar uma tupla

Como tuplas são imutáveis, métodos como `append()`, `pop()` ou `remove()` não existem.

```
tupla = (1, 2, 3)
```

```
tupla.append(4)
```

Isso gera erro:

```
AttributeError: 'tuple' object has no attribute 'append'
```

## 2 Conjuntos (Sets)

### 2.1 Para que servem?

Sets são estruturas usadas para armazenar elementos **sem repetição**.

Eles são úteis quando queremos:

- remover duplicatas
- verificar pertencimento rapidamente
- trabalhar com operações matemáticas de conjuntos

### 2.2 Sets são mutáveis ou imutáveis?

Sets são **mutáveis**.

Podemos adicionar ou remover elementos.

### 2.3 Criando sets

```
numeros = {1, 2, 3, 4}
```

```
print(numeros)
```

Também podemos usar a função `set()`.

```
numeros = set([1, 2, 3, 3, 4])
```

```
print(numeros)
```

```
# Saída: {1, 2, 3, 4}
```

Observe que valores repetidos são removidos automaticamente.

### 2.4 Adicionando elementos

```
numeros = {1, 2, 3}
```

```
numeros.add(4)
```

```
print(numeros)
```

```
# Saída: {1, 2, 3, 4}
```

## 3 Boolean e None

### 3.1 Boolean

Booleanos representam valores lógicos. Existem apenas dois valores possíveis:

- True
- False

Eles aparecem muito em comparações.

```
print(5 > 3)    # True
print(2 > 10)   # False
```

### 3.2 None

O valor None representa ausência de valor.

```
resultado = None

print(resultado)
# Saída: None
```

## 4 Dicionários

### 4.1 Para que servem?

Dicionários armazenam dados em formato de **chave e valor**. Eles são muito usados quando queremos associar informações. Exemplo:

- nome de aluno → nota
- produto → preço
- usuário → senha

### 4.2 Dicionários são mutáveis ou imutáveis?

Dicionários são **mutáveis**. Podemos alterar, adicionar ou remover itens.

### 4.3 Criando dicionários

```
aluno = {
    "nome": "Ana",
    "idade": 18,
    "curso": "Programação"
}

print(aluno)
```

### 4.4 Acessando itens

Para acessar valores usamos a chave.

```
print(aluno["nome"])
# Saída: Ana
```

## 4.5 Flexibilidade dos dicionários

Dicionários podem armazenar vários tipos de dados.

```
dados = {  
    "nome": "Carlos",  
    "idade": 20,  
    "notas": [7, 8, 9],  
    "aprovado": True  
}  
  
print(dados)
```

## 4.6 Alterando valores

```
aluno["idade"] = 19  
  
print(aluno)
```

## 4.7 Adicionando novos itens

```
aluno["cidade"] = "Vila Velha"  
  
print(aluno)
```

## 4.8 Métodos importantes

### 4.8.1 keys()

Mostra todas as chaves.

```
print(aluno.keys())
```

### 4.8.2 values()

Mostra todos os valores.

```
print(aluno.values())
```

### 4.8.3 items()

Mostra chaves e valores juntos.

```
print(aluno.items())
```

## Resumo Final

- Tuplas são imutáveis.
- Sets armazenam valores únicos.
- Booleanos representam verdadeiro ou falso.
- None representa ausência de valor.
- Dicionários armazenam dados em pares chave  $\rightarrow$  valor.

## 5 Erros Comuns com Tuplas, Sets e Dicionários

Ao aprender novos tipos de dados, alguns erros aparecem com frequência. Conhecer esses erros ajuda muito a evitá-los.

### 5.1 1. Tentar modificar uma tupla

Tuplas são **imutáveis**. Isso significa que seus valores não podem ser alterados.

**Exemplo errado:**

```
numeros = (10, 20, 30)
```

```
numeros[1] = 99
```

Isso gera erro porque não é permitido alterar elementos de uma tupla.

### 5.2 2. Usar métodos de lista em tuplas

Tuplas não possuem métodos como `append()`, `remove()` ou `pop()`.

```
tupla = (1, 2, 3)
```

```
tupla.append(4)
```

Erro:

```
AttributeError: 'tuple' object has no attribute 'append'
```

### 5.3 3. Esperar que sets mantenham ordem

Sets não garantem ordem dos elementos.

```
numeros = {3, 1, 2}
```

```
print(numeros)
```

A saída pode ser:

```
{1, 2, 3}
```

Ou em outra ordem dependendo da execução.

### 5.4 4. Colocar elementos repetidos em sets

Sets não armazenam valores duplicados.

```
numeros = {1, 2, 2, 3, 3}
```

```
print(numeros)
```

```
# Saída:
```

```
# {1, 2, 3}
```

### 5.5 5. Acessar dicionário como lista

Dicionários usam **chaves**, não índices numéricos.

**Errado:**

```
aluno = {"nome": "Ana", "idade": 18}
```

```
print(aluno[0])
```

**Correto:**

```
print(aluno["nome"])
```

## 5.6 6. Esquecer que chaves precisam existir

Se a chave não existir, ocorrerá erro.

```
aluno = {"nome": "Ana"}  
  
print(aluno["idade"])
```

Isso gera erro porque a chave "idade" não existe.

## 5.7 Resumo

- Tuplas são imutáveis.
- Sets não mantêm ordem e não permitem duplicatas.
- Dicionários usam chaves para acessar valores.
- Sempre verifique se a chave existe antes de acessá-la.

## 6 Comparação entre Estruturas de Dados

As estruturas de dados que aprendemos possuem características diferentes. A tabela abaixo resume as principais diferenças entre elas.

| Característica              | Lista   | Tupla   | Set     | Dicionário       |
|-----------------------------|---------|---------|---------|------------------|
| Usa colchetes ou símbolos   | [ ]     | ( )     | { }     | { }              |
| É mutável                   | Sim     | Não     | Sim     | Sim              |
| Mantém ordem dos elementos  | Sim     | Sim     | Não     | Sim              |
| Permite elementos repetidos | Sim     | Sim     | Não     | Não (nas chaves) |
| Possui índice numérico      | Sim     | Sim     | Não     | Não              |
| Usa chave e valor           | Não     | Não     | Não     | Sim              |
| Exemplo                     | [1,2,3] | (1,2,3) | {1,2,3} | {"nome": "Ana"}  |

### Resumo rápido

- **Lista:** coleção ordenada e mutável.
- **Tupla:** coleção ordenada e imutável.
- **Set:** coleção sem repetição e sem ordem definida.
- **Dicionário:** coleção de pares **chave** → **valor**.



## 7 Lista de Exercícios — Tuplas, Sets e Dicionários

### Nível 1 — Tuplas

1. Crie uma tupla chamada `cores` com três cores. Mostre o primeiro elemento.
2. Crie uma tupla com cinco números e mostre seu tamanho usando `len()`.
3. Crie uma tupla `(1,2,2,3,2)` e use `count()` para descobrir quantas vezes o número 2 aparece.
4. Crie uma tupla `(10,20,30,40)` e use `index()` para encontrar o índice do número 30.

### Nível 2 — Sets

1. Crie um set com os números `1,2,3,3,4,4`. Mostre o resultado.
2. Crie um set vazio e adicione três números usando `add()`.
3. Crie um set com nomes de três frutas e percorra esse set usando um `for`.
4. Verifique se o número 5 está dentro do set `{1,2,3,4,5}` usando o operador `in`.

### Nível 3 — Dicionários

1. Crie um dicionário chamado `aluno` contendo nome, idade e curso.
2. Mostre apenas o nome armazenado no dicionário.
3. Adicione uma nova chave chamada `cidade` ao dicionário.
4. Mostre:
  - todas as chaves
  - todos os valores
  - todos os pares chave-valor

## 8 Gabarito — Tuplas, Sets e Dicionários

### Nível 1

1.

```
cores = ("azul", "verde", "vermelho")

print(cores[0])
# Saída: azul
```

2.

```
numeros = (1,2,3,4,5)

print(len(numeros))
# Saída: 5
```

3.

```
tupla = (1,2,2,3,2)

print(tupla.count(2))
# Saída: 3
```

4.

```
tupla = (10,20,30,40)

print(tupla.index(30))
# Saída: 2
```

### Nível 2

1.

```
numeros = {1,2,3,3,4,4}

print(numeros)
# Saída: {1,2,3,4}
```

2.

```
numeros = set()

numeros.add(10)
numeros.add(20)
numeros.add(30)

print(numeros)
```

3.

```
frutas = {"maçã", "banana", "uva"}

for fruta in frutas:
    print(fruta)
```

4.

```
numeros = {1,2,3,4,5}

print(5 in numeros)
# Saída: True
```

## Nível 3

1.

```
aluno = {  
    "nome": "Ana",  
    "idade": 18,  
    "curso": "Python"  
}
```

2.

```
print(aluno["nome"])  
# Saída: Ana
```

3.

```
aluno["cidade"] = "Vila Velha"  
  
print(aluno)
```

4.

```
print(aluno.keys())  
  
print(aluno.values())  
  
print(aluno.items())
```